KAIROS BY EXOUSIA
FOUNDER HANDBOOK

EXOUSIA PRIME EMPORIUM LTD

Everything we have built, and why each part is shaped the way it is

The complete account. Not a summary and not a pitch — the reasoning behind every screen, in the order somebody new should meet it. Read it once end to end; come back to a single section when you need to change something in it.

19 AUGUST 2026 · PRE-LAUNCH · COUNTED FROM THE REPOSITORY

00 CONTENTS

01 THE THESIS

A day is a chain, not a list

Every calendar ever built stores appointments as independent rows. Move one row and one row moves. But an 11:00 only works because the 10:00 ends at 10:45 and the car takes twenty minutes — so when the 10:00 overruns, the 11:00 stays exactly where it was, sitting on top of the meeting that just ran over, the driver is still at the old time, and nothing anywhere says so. The principal finds out by being late.

That gap is the entire company. The work an assistant is paid for is the chain they run in their head, and no product in this category has ever modelled it.

Where the name comes from

Chronos is clock time — what a calendar stores. *Kairos* is the right moment. That is what an assistant is protecting when they move a meeting, and it is not a thing you can store in a row.

The second half of the job

Time is only half. The other half is *answers*: the passport number, the name exactly as printed, the BVN a bank asks for and the NIN a telco asks for. Those live today in a WhatsApp thread and an assistant's memory. Holding

them properly is the reason an office stays, once scheduling has got them in the door.

Why Nigeria first

Nigeria has the hardest version of every one of these problems. Traffic makes travel time the whole schedule rather than a rounding error. Identity is four numbers, not one, each demanded by name. A name board in an arrivals hall is a targeting notice rather than a courtesy. Build for that and London is a simplification; build for London and Lagos is a rewrite.

---

02 WHO IT IS FOR

Eight kinds of person, and each sees a different app

WHO	WHAT THEY GET
Principal	The account owner. Owns everything, decides who else may reach any of it. Lands on Today.
PA / EA / Chief of Staff	Full assistant remit: approvals, briefs, contacts, comms, itinerary. Lands on their own desk, with a switcher between principals. These are *titles, not tiers* — none outranks another.
Delegate	Scheduling only. A genuinely narrower remit, not a junior title. Sees availability and meeting types; never the vault's sensitive half.
Household staff	Driver, cook, housekeeper, nanny. Sees only what was addressed to them. Lives in its own table so no existing query can include them by accident.
Peer	An assistant at a different principal's office. A line of communication and nothing else — no reach into either principal.
Booker	Someone booking time from a public page. No account needed. Can reschedule or cancel through a link.

WHO	WHAT THEY GET
Driver at an airport	Holds a card link. Sees the flight, the meeting point, and a signal — never the principal's name.
Operator	Whoever runs the deployment. Reads `/api/status`, sets the environment, and is told plainly what is missing.

> WHY THIS MATTERS
>
> Chief of Staff is not a senior PA. Treating the three assistant titles as a hierarchy was the original mistake — a Chief of Staff was appointed as somebody's PA and read that word on every screen. The one genuinely hierarchical capability is that only a Chief of Staff can grant and withdraw other assistants' access to a space.

---

03 THE DAY
TODAY · ITINERARY

One screen that answers "what needs me"

Today is the landing screen. It merges bookings and itinerary items into a single ordered day and separates two questions the assistant actually asks: what is happening, and what needs a decision. Both come from one endpoint, so there is no way for the two halves to disagree.

Itinerary is the planner. An item has a start, an optional end, a kind, a location, travel minutes, and two flags that matter more than the rest: `is_anchor` and `status`.

Draft, proposed, confirmed

An assistant's draft is invisible to the principal. That is deliberate: the working copy of a day is not part of the day. *Proposed* puts it in front of the principal with a note; *confirmed* is the real day. The delay engine reasons over confirmed and proposed only.

The delay cascade

Say a thing overran by *n* minutes and Kairos computes what happens to everything after it — and shows that *before* applying it. Two rules do the

work:

> A gap absorbs.
>
> Each following item is measured against where the principal actually is, not against the original delay. The moment there is enough slack, the ripple stops and everything after it is reported unchanged. Saying "nothing needs doing" is worth as much as the warnings — a system that shunts the afternoon regardless cries wolf.

> An anchor does not move.
>
> A flight, a train, a court appearance. Kairos refuses to re-time it and reports how late you would be instead, in words: *you would reach this 20 min after it leaves. It will not wait.* Nothing after an anchor moves either — the anchor pins the rest of the day to its own clock.

The output is not a count. It is a list of effects with reasons, plus two deduplicated lists: **people to tell** (household staff, once each however many of their legs moved) and **attendees to tell** (external bookers). Neither is messaged automatically — the wording of "we are running late" is a judgement call, and always a person's.

Travel time with live traffic

`travel_minutes` has existed since the day was first modelled as a chain, and until recently it was a number somebody typed. In Lagos that number *is* the schedule: Ikoyi to Victoria Island is twelve minutes on a Sunday and eighty at six on a Thursday, and an assistant guessing once and reusing it forever is the commonest way a principal is late through nobody's fault.

So Kairos asks a maps provider at the departure hour, with traffic, and says which answer it got — a traffic-aware estimate or a plain one. Results are cached to the quarter hour. It never throws, and it **never silently rewrites the itinerary**: the estimate is offered and a person accepts it, because a schedule that reshuffles itself while nobody is looking is a schedule nobody trusts — and the assistant often knows something the road does not, like a closed gate or a convoy.

> PROVIDER SHAPE
>
> Google Distance Matrix today, behind an adapter with a base-URL override, exactly like the email service. Which maps API a deployment can get billing for is an operator's decision, not an architectural one.

---

04 BOOKING
AVAILABILITY · TIERS

Who may book you, and what that costs them

A principal sets weekly availability — several blocks per day, not one, because a real diary has a morning and an afternoon with a gap in the middle. Meeting types carry a duration, buffers before and after, a location or video, and an **access tier**.

TIER	WHO	WHAT HAPPENS
1	Anyone with the link	Books straight onto the diary
2	Anyone with the link	Books straight onto the diary
3	Named or invited	Goes to the approval queue
4	Inner circle	Goes to the approval queue

Tier 3 and above never lands on the calendar unapproved. It arrives in the **approval queue**, where the principal or a full assistant accepts, declines, or proposes another time. Tier drives the colour on the calendar too, so the shape of a week reads at a glance.

Slots

Open slots are computed from availability minus existing bookings and itinerary items, in the owner's timezone, over a fixed booking window. The same engine serves the public page, the assistant's view, and AI Assist — there is deliberately one implementation, because three would drift.

The public page

Lives at `/book/<handle>`, with an optional meeting-type slug. No account required. The booker gets a confirmation with a management link that lets

them reschedule or cancel without signing in — and rescheduling excludes their own booking from the slot computation, so their current time is offered back to them rather than appearing taken.

Video meeting types get an in-app join link. Email confirmations go through the same swappable delivery layer as everything else, and are recorded in the Outbox whether or not a provider is configured.

05 THE TEAM

Bringing somebody on, and taking them off

A **membership** connects an assistant to a principal, with a role and a status. Invitations go by email with a token; accepting one carries the invite context into onboarding so the new arrival knows whose desk they have joined before they have finished signing up.

Access to a PA route is granted to the principal themselves — a solo user can use their own approval queue without ever inviting anyone — or to a user with an active membership. Anyone else gets a 403.

Scheduling access is separately revocable

On by default for every assistant, because approving a Tier 3 booking while being unable to set the hours that produced the slot makes no sense. But some principals treat their own hours as personal, so it can be withdrawn per assistant. It is deliberately narrower than "can act as PA": profile, members and integrations stay owner-only whatever it says, because those are identity and access rather than scheduling.

Deleting an account

Most rows would go on their own — the schema declares `ON DELETE CASCADE` on everything hanging off a user by ownership. But a handful of columns reference a user *without* cascade: `created_by`, `author_id`, `assignee_id`, `decided_by`. Those record who did something rather than who owns it, and cascading there would delete a whole project because one contributor left.

So deletion walks them explicitly, authorship first and the user last, in one transaction. Anything missed surfaces as a foreign-key error rather than a silent partial delete, because a half-deleted account is far worse than a failed deletion.

06 PAIRING CODES

Bringing on an assistant without an email

The emailed invitation is still the better path when it works. But it needs a working mailbox and a configured provider, and a principal has no view of

either — so onboarding stalls on infrastructure nobody in the room can fix. A code read down the phone needs neither.

Three decisions carry its security, and all three are deliberate.

> ### The handle is required as well as the code.
> Two principals will eventually choose the same phrase, and a bare global code is a bearer token guessable against every account at once. Requiring the handle means an attacker must already know who they are targeting — and the collision problem disappears rather than being papered over with a uniqueness check that would itself leak which codes are live.

> ### It is armed, not standing.
> Off by default, live for a window the principal chooses, spent after a set number of joins. A credential to an executive's calendar that exists only in the hour it is needed cannot leak six months later out of an old message.

> ### Every failure answers identically.
> A wrong code, an unknown handle, an expired window and a spent code are indistinguishable from outside. Otherwise the error message becomes an oracle.

> SCOPE, DECIDED BY THE PRINCIPAL
>
> A code grants PA access to a principal's account and nothing else. It is not required to sign up — anyone may create an account freely, because an account on its own reaches nothing: there is no directory, no resolvable handle, no space and no membership. The door worth guarding is the one where somebody gains access to a *principal*, and that is the door the code is on.

---

07 ESSENTIALS
THE VAULT

The catalogue of things people are asked for and cannot recall

Not "documents" — *answers*. Every entry is a question somebody else puts to a principal: an airline, a hotel, a visa form, an event organiser. That is why the useful unit is a field with a copy button rather than a folder with a file in it.

CATEGORY	DEFAULT SENSITIVITY	EXAMPLES
Travel identity	SENSITIVE	Passport number and country, name as printed, nationality, visa, Known Traveller, driving licence, yellow fever card
Identity and registration numbers	SENSITIVE	BVN, NIN, voter's card, social security — and TIN and RC number, which are marked *ordinary* per field
Loyalty and memberships	ORDINARY	Airline programmes, hotel status, lounge access
Preferences	ORDINARY	Seat, meal, allergies, room
Sizes	ORDINARY	Suit, shirt, shoe
Insurance and emergencies	SENSITIVE	Policy numbers, next of kin, medical
Vehicles and addresses	ORDINARY	Plates, home and office addresses
Advisers	SENSITIVE	Lawyer, doctor, banker, accountant
Professional	ORDINARY	Titles, affiliations, bio lines

Sensitivity is per field, not per category

This is the load-bearing property, and the reason Nigerian identity numbers get their own group. A BVN is a key to somebody's banking and a delegate has no business seeing it. An RC number is printed on the company letterhead and withholding it would be theatre. One generic "National ID" field would have forced a choice between four numbers that are not interchangeable — a bank wants the BVN, a telco the NIN, an invoice the TIN — and a PA is asked for two of them in the same phone call.

Adding a field means deciding its sensitivity, once, in the catalogue. Not at each screen that happens to display it.

Encryption

AES-256-GCM, with the key held outside the database in `ENCRYPTION_KEY`. A stolen copy of the data is not a stolen identity — the attacker needs the running server's environment too. GCM rather than CBC because it authenticates as well as encrypts: a tampered ciphertext fails to open rather than decrypting to plausible rubbish. Stored as `v1:iv:tag:ciphertext`, base64, with the version prefix so a future rotation can tell old rows from new.

> THE BARGAIN, STATED PLAINLY
>
> Lose the key and the data is gone. There is no recovery path, by design — a recovery path is just a second key, usually a worse-kept one. The key is generated into the deployment's environment and backed up somewhere that is not this application. It never travels through chat or email, including to us.

With no key set the app runs normally and refuses only the sensitive fields, saying so. Turning the key on later is additive: nothing already stored breaks.

Expiry, trip readiness, and the travel block

Fields that expire carry a date and a computed state, warned well before the day. **Trip readiness** answers one question for a given travel date: is anything expired or expiring, and is the principal fit to travel — without ever putting the number itself in the response. The **travel block** assembles what an airline or a border will ask for, in one go, behind a step-up.

Documents can also be held on behalf of somebody else — a spouse, a child, a member of staff — which is what a family office actually needs and which the plan layer places at Executive.

---

08 STEP-UP
TWO-FACTOR

Proving it is still you, at the moment you ask for something sensitive

Revealing a passport number used to cost the account password. That defends against a borrowed laptop and against nothing else — because the attacker this vault has to survive is the one who *already knows the password*. Phished, reused, leaked: they sign in as you, and a gate made of the same password opens on the first try. Re-asking for a secret the intruder already holds is theatre.

So where two-factor is enrolled, the second gate asks for the **second factor**. Somebody who has taken the account still cannot read a BVN without the phone in your pocket. Where two-factor is not enrolled there is no second factor to ask for, so the password stands — the gate never gets weaker than it was.

Where the code is demanded is the principal's choice

And the default is *here*, not at sign-in. A code at the front door protects everything but is paid on every login — which is the friction that makes people turn two-factor off, and an account with two-factor off protects nothing at all. Spending it on the vault instead puts the cost where the value is: a stranger with the password reaches a calendar, and still cannot read a passport number. A principal who wants it at both is one switch away.

A short grace window

One step-up covers a few minutes of work. Without it, a Chief of Staff at a check-in desk reads a passport, then a visa, then a Known Traveller number, and types three codes from an app that rotates every thirty seconds — so at least one means standing there waiting for the next. Friction that heavy does not make people careful; it makes them turn two-factor off. The window is short, per-session, and every reveal inside it is still logged individually.

The rest of the perimeter

- **TOTP** to RFC 6238, written out rather than pulled from a package — it is thirty lines of HMAC and base32, and a dependency in the authentication path is a dependency you have to trust forever.
- **Recovery codes**, each usable once.
- **Rate limiting** keyed on both the account being attacked and the address attacking, so neither a single account being hammered nor one source spraying many accounts gets through. In memory, which is honest about its limits: it resets on restart and does not span instances.
- **Sessions** as cookies, with scrypt password hashing.
- **An access log**: every reveal of a sensitive field, recorded.

---

09 TRIPS

A journey as an object, and the two things that follow

First: the principal's day is drawn where they are.

Today used to compute the whole day in the timezone on the principal's profile — their home zone, always. A week in London was therefore rendered in Lagos time: "today" began and ended at the wrong moment, a 09:00 meeting showed as 08:00, and the cascade reasoned about gaps against the wrong wall clock. The itinerary row could always express a leg that crosses zones; nothing read those columns, because nothing knew where the principal was on a given date. A trip knows.

Second: a leg can say who is meeting the principal.

An itinerary item could name exactly one person — the household driver. That is a household relation and by definition exists only at home. Land in London and the arrival transfer has a person-shaped hole. So a leg carries an *arrangement*:

ARRANGEMENT	NEEDS A CONTACT	WHY IT EXISTS
`own_driver`	No	Your household driver. Only at home.

ARRANGEMENT	NEEDS A CONTACT	WHY IT EXISTS
`hired`	Yes	A car service — needs a booking reference and a dispatcher to call.
`hotel`	Yes	Arranged by the hotel. They need the flight number, or nobody comes.
`host`	Yes	The office or host at the other end is sending someone.
`own_way`	No	A decision, recorded — so nobody is improvising in an arrivals hall.

The journey builder

A trip is built from the screen that shows it. The chain form takes a flight, both cars, the hotel check-out and the transfer, and the armed pickup, in one pass — because that is how an assistant thinks about an arrival, not as five unrelated rows. There is also a single-leg form for everything else.

Times are entered as wall-clock in the leg's own zone and converted at the boundary. A searchable timezone picker covers every zone the browser knows, ranked so that typing "lon" offers London before Longyearbyen, with aliases for the cities people actually name — Abuja resolves to Africa/Lagos.

Trips also carry **travellers** (who else is on this journey) and **contacts** (who to call at the other end).

---

10 MEETING A CAR

Neither side holds up a name

The ordinary way an executive is met at an airport is a stranger holding a placard with their name on it. That board publishes, to everyone in an arrivals hall, that this named person has just landed and is about to get into a car. For a principal in this market that is not an indignity — it is a targeting notice, and the people who read placards for a living are exactly the audience it is worst to have.

The phrase

Both sides hold the same short phrase, agreed in advance. The principal knows the car, the plate and the phrase. The driver knows the flight, the meeting point and the phrase. Whoever speaks first, the other answers, and nobody's name is said aloud. It is **per pickup** — a phrase that stayed the same across trips would end up known by every driver a household has ever used, which is a standing credential with a rotation problem.

The signal

The phrase solves half an arrivals hall. It only works once two people are already facing each other — and the principal walks out of customs into forty strangers holding forty boards with nothing to look for. So both screens show the same thing at the same second: a plain colour and a plain shape, derived from the card token by HMAC over a rotating window.

The driver holds the phone up, screen outward. The principal is looking for orange with a triangle on it, not for their own name in a stranger's hands. It is a name board with the name taken out — which is the only part of a name board that was ever a problem.

Once the driver marks that they have been found, the signal **freezes**, so it stays true while the principal walks over rather than rotating out from under them.

The driver's card lives at `/pickup/<token>` and needs no account. It shows the flight, the meeting point, the phrase and the signal — and never the principal's name.

---

11 VISAS

The part that can be answered truthfully, and the part that cannot

"Visa requirements" is two questions wearing one name, and they are not equally safe to answer.

> ### Coverage — does the visa I hold cover this trip?
>
> Deterministic, from facts the principal supplied: a Schengen multi-entry valid to March, a single-entry US visa already used in January, a UK visa that expires four days before the return flight. Nothing has to be looked up, and being wrong in a way that matters is impossible, because the inputs are the principal's own documents. **This is built.**

> ### Requirement — does a Nigerian passport need a visa for Kenya?
>
> Forty thousand nationality-by-destination pairs, revised by governments without notice, where a wrong "no visa needed" strands somebody at a check-in desk with a boarding pass they cannot use. Not guessable and not responsibly scrapeable. It stays a provider adapter, unconfigured, and the screen says so. **This is refused.**

Coverage returns one of six states, and the useful one is the third:

STATE	MEANS
`covers`	Valid for the whole trip, entries remaining.
`none`	Nothing on file for that country.
`expired`	Already expired before departure.
`expires`	Valid on the way out and not on the way back. The failure nobody catches by eye.
`not_yet`	Valid, but not until after this trip starts.
`spent`	Entries used up.

Processing-time guidance is included and carries a review date on its face, so nobody relies on a figure that has quietly gone stale.

> THE NUMBER IS NOT IN THIS TABLE
>
> A visa's identifying number stays in the essentials vault, encrypted and behind a step-up. Coverage records what a visa covers, not what it is.

12 THE HOUSEHOLD

A driver, a cook, a housekeeper, a nanny

Deliberately *not* a role on memberships. Access to a PA route is granted on any active membership whatever the role, and five other queries do the same — so a household role added to that table would have handed a cook the approval queue, contacts, briefs and the ordinary tier of the vault at six call sites at once, none of which would have looked wrong in review.

A separate table means no existing query can include them by accident. Same reasoning as private spaces: structural, rather than a flag somebody could flip later without noticing what it touched.

`job_title` is free text and carries no access whatever. A driver and a chef see exactly the same thing: what was addressed to them. Instructions go out, replies come back, and an itinerary leg can name a member of staff so the cascade knows who to tell.

Hiring and dismissing is owner-only — the same kind of act as appointing an assistant, because it decides who is inside the household.

---

13 THE DIRECT LINE

The room that is already there

Spaces, threads and messages already existed, and a principal *could* create a space and invite their assistants into it. Nobody does. The message that needs sending — "car's outside", "he's running twenty minutes late", "can you confirm the Thursday dinner" — is worth thirty seconds, not a setup flow, so it ends up on WhatsApp and out of the record entirely.

So every principal has exactly one direct line, created the moment they have somebody to talk to, containing them and every active assistant. One room rather than a DM per pair, because that is the actual shape of the work: two assistants covering the same principal need to see each other's traffic, not run parallel private conversations about the same diary.

It is an ordinary space and an ordinary thread, so notes, records, acknowledgement and task-from-message all work here for free.

Voice

Three decisions carry it, and each is about a recording being *more* sensitive than the typed message it replaces, not less.

- **It requires the encryption key** — the same key the vault requires. Without one it says so and refuses. Holding a principal's voice in plaintext because a key was inconvenient to set is the one outcome worth refusing outright.
- **It expires**, thirty days by default. An operational note has a useful life measured in hours, and audio kept past its usefulness is a liability nobody revisits until it leaks.
- **It is capped hard** — two minutes, two megabytes, and only the formats browsers actually produce. A direct line is for "the car is downstairs", not for dictating a memo, and an uncapped audio endpoint is an invitation to fill somebody else's database.

The assistant can also dictate typed messages using the browser's own speech recognition, with an explicit statement on screen about where that audio goes.

Marking an instruction carried out

A voice note that says "book the Thursday dinner" leaves no trace of having been acted on. So a *note* can be marked done, by anybody in the room, and un-marked. A *record* cannot: a record is acknowledged, not carried out — and saying so in the refusal is how the distinction stays learnable.

---

14 SPACES
PROJECTS · TASKS

Compartments, and two registers of message

A space has a **context**: work, personal, or private. Role sets the opening position rather than a ceiling — assistants are auto-granted work spaces, personal is opt-in because a PA routinely runs household logistics, and private is unreachable for everyone including a Chief of Staff.

Every read and write of a space, thread or message goes through one access resolver. There is deliberately no second path, because isolation rules are only worth anything if they cannot be routed around. Two properties matter:

- A space you cannot see returns **nothing**, indistinguishable from one that does not exist. The mere existence of a space is information the owner

has not shared.

- A private space **short-circuits before consulting membership at all**, so a stray row from a migration, a bug or a future refactor still cannot open a door that is supposed not to exist.

Notes and records

Two registers in the same thread. A **note** is conversation: editable, deletable, ordinary. A **record** is a commitment — a decision, an approval, a blocker, a sign-off — and is immutable once filed. A note can be *promoted* to a record, which is the moment a conversation becomes a fact.

> That distinction is the reason an office would move off WhatsApp at all.

Projects and stages

A project has ordered stages, and a stage's status is *stored* so an owner can set it by hand and have that stick — but **records override it**. Any open blocker makes a stage blocked; otherwise an accepted sign-off makes it done; otherwise, if it was auto-moved, it returns to active; otherwise it is left alone.

A blocker everyone can see while the board still says "active" is exactly the disconnect this product exists to remove. Filing a record is what actually moves the project — which is precisely what makes promoting a note worth doing.

Tasks and reminders

A task can be created straight from a message, assigned, given a due date, and closed. Each task and stage carries a reminder stage (`null → due_soon → overdue`) so a nudge fires exactly once per threshold rather than on every sweep. Changing a due date, reassigning, or reopening clears it, because those genuinely deserve a fresh set.

How much warning something gets scales with what missing it costs. Reminders go through the same email service as everything else, so with no provider configured they still land in the Outbox and the server log — visible and testable rather than silently dropped.

---

15 THE DESK

What an assistant gets that a principal does not

Assistants land on their own home with a switcher across principals. Everything below is scoped to whichever principal is selected.

Approvals

Tier 3 and 4 bookings waiting on a decision. Accept, decline, or propose another time.

Contacts

Contact intelligence: who this person is to the principal, relationship tier, history, and the dates that matter.

Relationships

Birthdays and anniversaries as a rolling calendar, counted forward from today so nothing is missed by a week.

Briefs

A pre-meeting brief: who is in the room, what was last agreed, what to raise. Can be pre-filled from what the app already holds.

Instructions

The principal's standing instructions, written once, readable by whoever is covering.

Comms

Drafting and sending on the principal's behalf, with every message recorded in the Outbox.

AI Assist

Plain-language scheduling: "find Adaeze an hour next week". Extracts a contact, a meeting type and a date hint, then filters real open slots.

Itinerary

The day planner, drafts included — the assistant's working copy, invisible to the principal until proposed.

> AI ASSIST NEVER BOOKS ANYTHING
>
> It returns candidates. Every one still needs an explicit click from a person. That is the blueprint's rule — the assistant approves every output, never autonomous — and it holds whether the extraction is done by the current pattern parser or by a model call later. The contract does not change.

16 PEERS
HANDLES

Two assistants, two different executives, one room to book

That negotiation is the single most common conversation in the job, and the app had nowhere to hold it: the two share no principal, no space and no membership, so nothing in the model connected them. It went to WhatsApp, and the confirmation went with it.

A **connection** is emphatically not a delegation. It gives neither side any reach into the other's principal, calendar, contacts or team. It is a line of communication and nothing else, which is why it lives in its own table: there is no query anywhere that could mistake one for the other.

Handles

A person is `@ada`, and their booking page happens to live at `/book/@ada` — not the other way round. Handles are deliberately **not a global directory**. Kairos is built on compartments, and a handle that resolved for anyone would quietly undo that. Look one up that you have no relationship with and you get nothing back, indistinguishable from a typo.

A handle resolves when you already support each other, share a space, or have an accepted peer connection. Establishing a new relationship still goes through an invitation, because that is an act and should feel like one. A reserved list keeps the app's own paths and the names people would impersonate off the market.

17 NOTICES

Who may write to everyone

Read from the environment, not from a column. There is deliberately no way to grant yourself this from inside the app and nothing in the database to flip — the same reasoning as the encryption key living outside the data it protects.

Unset means nobody can post, and the screen says so — to the people who could act on it, and to nobody else. Notices can be addressed to everyone or to assistants only, arrive with an unread badge on the nav rail, and are marked read by opening the page.

18 HONESTY
THE CAPABILITY REGISTRY

Saying what does not work, in the place it will work

The first version of this put a list at the foot of each screen. That is a footnote, and a footnote is not the same as seeing the feature: it tells somebody a thing is planned without ever showing them where it will live or what it will be called.

So each unbuilt capability now stands in the place the working one will occupy, named the way it will be named, and visibly inert. **Pressing it does something** — it opens a line saying it is not ready and what it is waiting on. A control that is merely greyed out and swallows clicks reads as broken, and somebody being shown the product cannot tell a disabled button from a bug.

One registry decides all of it, and computes availability from the same environment the feature itself reads. Set the maps key and the travel-time placeholder disappears by itself, because the notice and the feature are asking the same question. There is no screen anywhere that decides for itself whether something works.

Two states, said differently

STATE	MEANS	READ BY
soon	The work is ours — code to write, or a contract to sign.	A principal, being asked to wait

STATE	MEANS	READ BY
`needs_key`	The work is done; this deployment is missing something specific.	An operator, being handed a task

There is also a `/coming` screen listing every one of them with its screen, its control name and the variable it awaits — plus what *is* working on this deployment. A capability drops off that page and its placeholder disappears in the same instant, from one fact on the server.

The two refusals

Concierge and visa requirement are not backlog. The concierge desk is gated on *people* — a vetted network who answer at 2am in the city the principal is actually in — so nothing on that screen offers to "connect", because there is no key that turns people on. And the screen has no request box: the obvious placeholder, a form that takes a request and returns a friendly message, would be a lie told to somebody at the exact moment they were relying on us. The one thing it accepts is an expression of interest, which is real, is kept, and says on its face that it reaches Exousia rather than a concierge.

---

19 PLANS
CONNECTORS

What a plan includes, and — until launch — what it would have excluded

The word "tier" is deliberately not used for this. It already means a meeting type's access tier and a contact's relationship tier; a third meaning would make every conversation about any of them ambiguous. The commercial one is a **plan**.

PLAN	PER MONTH	ADDS
Free	₦0	Your own diary, bookings, essentials at ordinary sensitivity
Standard	₦8,000	An assistant, the full vault, contacts, tasks, the direct line

PLAN	PER MONTH	ADDS
Plus	₦15,000	Spaces and projects, trips, voice notes, travel time, AI Assist, household
Executive	₦25,000	Briefs, peer connections, documents held for others, the concierge desk
Family Office	₦120,000	More than one principal
Enterprise	On application	Single sign-on

Two rules that are not negotiable

> ### Entitlement is never access control.
> Whether a delegate may see a BVN is decided by the essentials catalogue and a second factor. Whether the account is on Plus is decided by the plan layer. If those ever merge, a billing bug becomes a data breach, and a lapsed card locks a principal out of their own passport — precisely the thing this product exists not to do.

> ### It fails open.
> A null column, a half-run migration, an unknown plan name: the answer is allow. The failure mode of a strict check is somebody at an airport unable to read their own visa number because a database default did not apply. A month of unpaid Plus costs less than that, every time.

And it ships before billing does, switched off. Every account is on `founding`, which grants Executive. What the layer does meanwhile is record which plan each action *would* have needed, so the pricing question is answered by evidence when the switch is thrown. Gating applies to creating things, never to reading them.

Seventeen connectors, two kinds

A **deployment** connector is configured once by whoever runs Kairos and no principal ever sees a button for it — flight data, object storage, the

transcription route. On for everybody or off for everybody. An **account** connector is connected by each principal to their own thing: their Google Calendar, their WhatsApp number, their Zoom. Two states must both be true, and telling them apart is the difference between "we haven't built this" and "you haven't finished setting it up" — very different sentences to read.

All seventeen are catalogued, placed on a plan, and currently return *not implemented*. The catalogue exists; the integrations do not. Do not describe them as available.

---

20 RUNNING IT

What the operator has to know

One web service. In production the server also serves the built client and hands unknown paths back to the app for client-side routing.

The server comes up and explains itself

It binds its port immediately and reports readiness, rather than refusing to start until the database answers. The old order made a database problem look like a deploy problem: nothing listened, so the platform showed a successful build followed by a deploy spinning forever with no explanation. Every API route is still held at 503 until the schema is ready — what changed is that the failure is now visible at a URL instead of invisible in a queue. The page itself always loads, because a blank 503 tells a person nothing.

The status endpoint

`/api/status` reports whether storage is durable, whether the database is ready, which backend is in use, the named target (never the password — it is public), and whether email delivery is configured.

Storage is the one that bites

A hosted web instance has an ephemeral filesystem. A local database file there is wiped on every restart, taking every account with it — which arrives at the login screen as "incorrect email or password" and looks nothing like a storage problem. Setting a Postgres connection string switches backends and the data survives. Until it is set the app still runs, and the login screen says so after a failed sign-in rather than blaming your password.

Environment, in plain terms

SETTING	WHAT IT TURNS ON
`DATABASE_URL`	Durable storage. The single most important one.
`DATABASE_SCHEMA`	Keeps Kairos in its own schema when sharing a database with another app.
`ENCRYPTION_KEY`	Sensitive fields, two-factor, voice notes. 32 bytes. No recovery path.
`ANNOUNCEMENT_AUTHORS`	Who may publish notices. Unset means nobody.
`Email provider key`	Real delivery. Without it every message still lands in the Outbox.
`MAPS_API_KEY`	Travel time with live traffic.
`DEFAULT_PLAN · PLAN_ENFORCEMENT`	Which plan new accounts get, and whether plans are enforced at all.

> HOW CREDENTIALS TRAVEL
>
> Connection strings, provider keys and the encryption key go from the provider or the generator *straight into the deployment's environment*. Never through chat, never through email, never into a document — including to us. The key card in the app says exactly this, and it is not a formality.

21 OUTSTANDING

What is not built, and why not

THING	WHY IT IS NOT DONE	
Live flight status	Needs a flight-data contract. The delay cascade is already shaped to consume it.	CONTRACT
Forward a confirmation	Needs an inbound mail domain and a parser for airline and hotel mail.	BUILD
Attach a scan	Needs object storage. The passport page itself, encrypted, when the number alone is not enough.	BUILD
Transcribe a voice note	Needs a route that does not hand the audio to a third party. When it lands, the text becomes the durable artifact and the thirty-day expiry stops costing anything.	BUILD
Whether a visa is required	Refused until there is a maintained ruleset. A wrong "no" strands somebody.	REFUSED
The concierge desk	Contracted, vetted people. There is no key that turns this on.	PEOPLE
Calendar sync, WhatsApp, Zoom, Teams, SMS, and the rest	Catalogued and placed on plans; every one currently returns not-implemented.	CREDENTIAL
Billing	The plan layer exists and is switched off. There is no payment integration.	BUILD

Known operational limits

- Rate limiting is in memory: it resets on restart and does not span instances. Adequate for one instance; it needs a shared store before running several.
- Free hosted Postgres has no backups. Fine while evaluating; move to a paid instance before this holds a real principal's calendar and contacts.
- The app has not been through an external security review.
- No principal is paying yet. Every claim about behaviour in this handbook is a claim about the software, not about usage.

22 THE RULES

Things we do not do, whatever the reason looks like

1. **Never claim a feature works when it does not.** Not in a demo, not in a deck, not in a screen. This is also our best sales line.
2. **404, not 403,** for anything whose existence is itself private.
3. **Prefer structural to conditional.** If a door can be made not to exist, make it not exist rather than locking it.
4. **Never move an anchor.** Not to make a schedule work, not to avoid an awkward message.
5. **Never send on somebody's behalf automatically.** Draft it; a person sends it.
6. **Never guess an answer somebody will act on at a border.** Say we do not know.
7. **Entitlement never gates reading.** Money problems must not become access problems.
8. **Credentials never travel through a conversation.** Including with us.
9. **A record is immutable.** If that becomes inconvenient, the answer is a new record, not an edit.
10. **Sensitivity is decided once, in the catalogue.** Never at the screen.

> We would rather ship less and be believed about all of it.